



Documentação do Sistema de Design & Overlays

Este ecossistema foi projetado para ser **minimalista, performático e visualmente limpo**. Ele utiliza um sistema híbrido de tokens de CSS nativos injetados no Tailwind v4, um manipulador de portais imune a quebras de renderização no servidor (SSR) e um sistema de notificações leve.



1. Sistema de Temas e Cores (**global.css**)

O tema centraliza a identidade visual capturada na sua interface original, utilizando fundo neutro ultra-leve no aplicativo para fazer com que os blocos de conteúdo puramente brancos saltem aos olhos.

Variáveis de Ambiente CSS (**:root**)

Se precisar alterar qualquer tom da plataforma globalmente, altere estes valores no **global.css**:

V ar iá v el	C o r (L i g h t)	C o r (D a r k)	Aplicação Primária
-- b a c k gr o u n d	# f 8 f a f c	# 0 9 0 9 0 b	Tela de fundo geral do app

-- pr i m ar y	# 6 C 4 D E 6	# 7 C 6 2 E 9	Botões principais, links ativos e ícones
-- d ar k- tit le	# 1 B 1 4 6 4	# E 0 E 7 F F	Títulos marcantes e cabeçalhos
-- m ut e d	# 8 F 9 2 A 1	# A 1 A 1 A A	Textos secundários, placeholders e labels
-- b or d er	# e 2 e 8 f 0	# 2 7 2 7 2 A	Linhas divisórias e bordas sutis
-- c ar d- b g	# f f f f f	# 1 8 1 8 1 b	Cards, Modais, Drawers e Formulários

Classes Utilitárias Prontas

Para manter a velocidade de desenvolvimento, use as classes estruturais predefinidas em vez de digitar múltiplas classes utilitárias toda vez:

- **.ui-card**: Bloco com bordas suavizadas (16px), fundo branco e sombra quase imperceptível.
- **.ui-modal**: Recipiente flutuante centralizado com cantos em 20px e sombra projetada suave.
- **.ui-input**: Input minimalista sem caixa, contendo apenas a linha inferior que reage ao foco com a cor roxa principal.
- **.title-purple**: Aplica a tipografia pesada com espaçamento de letras reduzido (-0.02em) no tom escuro da identidade.

Padrão de Botões (.btn)

Os botões seguem o formato "pílula" (rounded-full) e possuem uma micro-interação premium de clique (scale(0.97)).

HTML

```
<button class="btn btn-primary">Ação Principal</button>
```

```
<button class="btn btn-secondary">Ação Secundária</button>
```

```
<button class="btn btn-outline">Voltar</button>
```

```
<button class="btn btn-ghost">Saiba mais</button>
```



2. O Utilitário de Portal (Portal.jsx)

O componente `<Portal>` resolve o problema clássico de conflito de `z-index` e quebras de layout herdadas de elementos pais. Ele teleporta componentes pesados para fora do fluxo do HTML principal.

⚠ Nota de Arquitetura: Ele foi desenvolvido com uma trava de montagem (`mounted`) para evitar erros de hidratação em frameworks como Next.js/Remix, garantindo que o portal só execute no navegador do cliente.

Como usar:

Basta encapsular o elemento flutuante (Modais, Drawers, Comboboxes customizados) dentro da tag `<Portal>`:

JavaScript

```
import Portal from '@components/Portal';

export function MeuComponenteFlutuante() {
  return (
    <Portal>

      <div className="fixed inset-0 bg-black/10 backdrop-blur-xs z-50">

        {/* Seu Modal ou Drawer entra aqui */}

      </div>

    </Portal>

  );
}
```

3. Contexto de Notificações (`ToastContext.jsx`)

Gerencia alertas temporários no canto inferior direito da tela. Os toasts são empilháveis, limpos e desaparecem sozinhos após 3,2 segundos.

Configuração Inicial

Certifique-se de que o `ToastProvider` encapsule a árvore de componentes no seu arquivo de layout global:

JavaScript

```
// src/app/layout.jsx ou src/main.jsx

import { ToastProvider } from '@contexts/ToastContext';

export default function RootLayout({ children }) {
```

```

return (
  <html>
    <body>
      <ToastProvider>
        {children}
      </ToastProvider>
      <div id="portal-root" /> {/* <--- Crucial para o funcionamento do Portal */}
    </body>
  </html>
);
}

```

Disparando Alertas nos Componentes

Importe o hook personalizado `useToast` e utilize a função `addToast(mensagem, tipo)`:

JavaScript

```

import { useToast } from '@contexts/ToastContext';

export default function PaginaConfiguracao() {
  const { addToast } = useToast();

  return (
    <div className="flex gap-4">
      {/* Toast Padrão (Sucesso / Informativo) */}
      <button
        className="btn btn-primary"
        onClick={() => addToast('Perfil atualizado com sucesso!')}
      >

```

```

        Salvar

    </button>

    { /* Toast de Erro */ }

    <button

        className="btn btn-outline"

        onClick={() => addToast('Falha ao conectar ao servidor.', 'error')}

    >

        Simular Erro

    </button>

</div>

);

}

```

4. Caso de Uso Real: Juntando Tudo (Exemplo de Modal)

Veja como criar um Modal minimalista perfeito que utiliza o **Tema**, o **Portal** e dispara um **Toast** ao concluir a ação:

```

JavaScript

import React from 'react';

import Portal from '@components/Portal';

import { useToast } from '@contexts/ToastContext';

export function ModalConfirmacao({ isOpen, onClose }) {

    const { addToast } = useToast();

```

```
if (!isOpen) return null;
```

```
const handleConfirmar = () => {
```

```
  // Código da sua regra de negócio aqui...
```

```
  addToast('Operação realizada com absoluto sucesso!');
```

```
  onClose();
```

```
};
```

```
return (
```

```
  <Portal>
```

```
    {/* Backdrop de fundo escurecido sutil */}
```

```
    <div className="fixed inset-0 bg-black/20 dark:bg-black/50  
    backdrop-blur-xs z-50 flex items-center justify-center p-4">
```

```
      {/* Estrutura do Modal (Fundo Branco Puro Clean) */}
```

```
      <div className="ui-modal w-full max-w-sm p-6 animate-in fade-in  
      zoom-in-95 duration-200">
```

```
        <h3 className="title-purple text-lg mb-2">Confirmar Ação</h3>
```

```
        <p className="text-sm text-[var(--muted)] mb-6">
```

```
          Você tem certeza que deseja prosseguir? Esta ação não poderá ser  
          desfeita.
```

```
        </p>
```

```
        {/* Grupo de botões minimalistas */}
```

```
        <div className="flex justify-end gap-2">
```

```
          <button className="btn btn-ghost" onClick={onClose}>
```

```
            Cancelar
```

```
</button>
```

```
<button className="btn btn-primary" onClick={handleConfirmar}>
```

```
  Confirmar
```

```
</button>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</Portal>
```

```
);
```

```
}
```